14.3.2 In-memory caching with IMemoryCache

Remember back in chapter 4 when you created a custom route constraint? It compares route values with the contents of a hardcoded list to determine whether a route matches. I mentioned at the time that this should perform a lookup against the contents of the database rather than a hardcoded list, which would potentially need constant updating. Implementing this as a constraint is problematic in that constraints are synchronous; any calls to asynchronous services must be made synchronously. You could move this task to middleware instead, or you could move the task to the actual page that uses the lookup: the City page. In this section, that's what you'll do, and you'll use the `IMemoryCache` service to cache the names of the cities, so the database lookup is only performed when necessary.

The `IMemoryCache` service is enabled as part of Razor Pages. The interface defines three methods for managing cache entries: `CreateEntry`, `Remove`, and `TryGetValue`. However, for getting and setting cache entries, you are more likely to use one of a number of extension methods instead, which have been designed to be simple to use. These are detailed in table 14.2.

Table 14.2 `IMemoryCache` extension methods